

java-ddd-clean-26

Arquitectura del monolito modular — Modular monolith architecture

Spring Boot 4 · JDK 26 (bytecode 25) · Gradle 9.7 multi-módulo · PostgreSQL · Kafka

Documento bilingüe: español (pp. 1–5) · English (from p. 6)

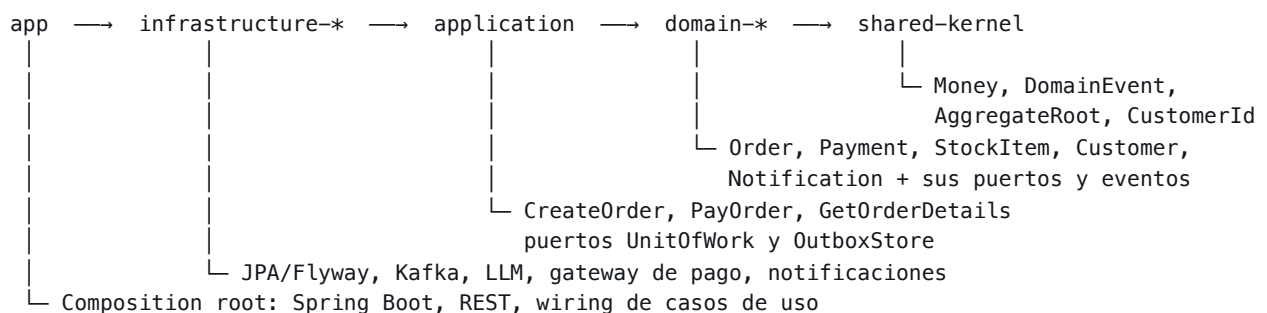
ESPAÑOL

1. Qué es este sistema

Un monolito modular de comercio electrónico organizado en **catorce módulos Gradle**. La tesis del diseño es que cada bounded context pueda extraerse como microservicio sin reescribir su lógica: para eso el dominio no conoce ningún framework, todo acceso al exterior pasa por puertos, y los contextos se comunican por eventos, nunca por llamadas directas.

Lo que distingue a este repositorio de un diagrama bonito es que **esas fronteras las verifica el build**: veinticinco reglas ArchUnit fallan la compilación si alguien las cruza.

2. Mapa de módulos y regla de dependencias



Las flechas van en una sola dirección. `domain-*` no importa Spring, ni JPA, ni Jakarta Validation; `application` tampoco. El framework vive exclusivamente en `app` y en `infrastructure-*`.

Por qué la capa de aplicación tampoco conoce Spring

Es la decisión menos habitual del repositorio y la que más consecuencias tiene. Un caso de uso necesita atomicidad, y el reflejo normal es anotarlo con `@Transactional`. Eso ataría la capa de aplicación al framework. En su lugar la transacción entra por un puerto:

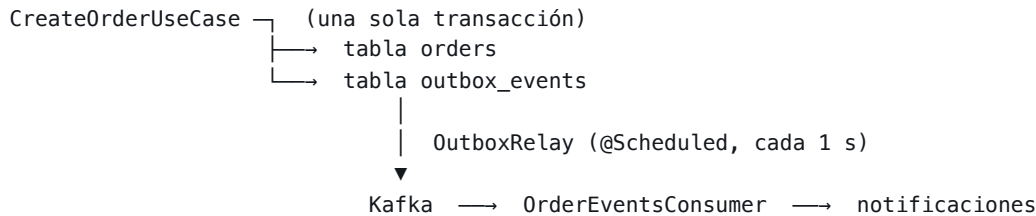
```
public interface UnitOfWork {
    <T> T execute(Supplier<T> work);
}
```

La infraestructura lo implementa con `TransactionTemplate`. El caso de uso obtiene la atomicidad sin importar una sola clase de Spring, y la regla ArchUnit `applicationLayerDoesNotDependOnSpring` impide que eso se degrade con el tiempo.

3. El núcleo: cómo se garantiza la consistencia

3.1 Outbox transaccional

Un evento de dominio **nunca se publica directamente a Kafka**. El caso de uso escribe el agregado y su evento en el mismo commit; un relay los publica después.



El problema que resuelve. Guardar en base de datos y publicar en Kafka son dos sistemas distintos. Si se hacen por separado —el patrón *dual-write*— y Kafka falla justo después del commit, la orden existe pero el evento se perdió para siempre, sin traza ni forma de detectarlo. Con el outbox el evento está en la misma transacción que la orden: o se guardan los dos o ninguno.

El relay solo marca `published = true` cuando el broker confirma la escritura. Si Kafka está caído, la fila sigue pendiente y se reintenta en el siguiente ciclo. Esto entrega garantía **at-least-once**, y por eso el consumidor deduplica por `eventId`.

El publisher está anotado `@Transactional(propagation = MANDATORY)` de forma deliberada: si alguien intenta publicar un evento fuera de una transacción activa, la llamada falla en vez de reintroducir silenciosamente el *dual-write*.

3.2 Idempotencia en pagos

Cobrar es la operación menos reintentable del sistema. El endpoint exige cabecera `Idempotency-Key` y el caso de uso se divide en tres pasos:

PASO	QUÉ OCURRE
Transacción 1	Se reserva la clave creando el pago en estado <code>PENDING</code> . La unicidad la garantiza el índice de <code>payments.idempotency_key</code> , no el código.
Sin transacción	Se llama al gateway de pago. Una llamada de red nunca debe mantener abierta una transacción de base de datos.
Transacción 2	Se registra el resultado y el nuevo estado de la orden en un solo commit.

```
curl -X POST localhost:8080/api/v1/orders/$ORDER_ID/payments \
-H "Idempotency-Key: $(uuidgen)"
```

Repetir la petición con la misma clave devuelve el pago existente sin volver a llamar al gateway. Sin esto, un reintento del cliente o del balanceador cobraba dos veces.

4. Reglas de arquitectura ejecutables

REGLA	QUÉ IMPIDE
<code>domainDoesNotDependOnSpring</code>	Que el dominio quede atado al framework y no pueda extraerse.
<code>applicationLayerDoesNotDependOnSpring</code>	Que los casos de uso se llenen de anotaciones en lugar de puertos.
<code>controllersDoNotAccessRepositories</code>	Que un controlador salte el caso de uso y hable con la base de datos.
<code>onlyInfrastructureImplementsPorts</code>	Que un adaptador aparezca fuera de la capa de infraestructura.
<code>noCyclicDependenciesBetweenModules</code>	Ciclos entre módulos, que harían imposible extraer uno solo.
<code>domainModulesDoNotDependOnEachOther</code>	Acoplamiento directo entre bounded contexts. Cubre los 20 pares .

Una regla que pasa en vacío es peor que no tenerla. El import de ArchUnit falla de forma explícita si no logra analizar ninguna clase, precisamente porque ese fue el fallo silencioso más grave que tuvo este repositorio.

5. Decisión: JDK 26 con bytecode 25

El build **compila con el JDK 26** (toolchain auto-aprovisionado vía foojay) pero emite **bytecode 25** mediante `options.release = 25`.

El motivo es concreto y medible: el ASM que empaqueta ArchUnit 1.4.1 solo lee hasta class file **V25**. Con bytecode V26 descarta todas las clases *en silencio* —sin error, sin aviso— y las reglas de arquitectura pasan sin comprobar nada. Se prefirió tener las reglas funcionando a tener el número de versión más alto en el bytecode. La decisión se revisa en cada subida de ArchUnit.

Por la misma razón el código de producción no usa APIs preview: `GetOrderDetailsUseCase` paraleliza con `Executors.newVirtualThreadPerTaskExecutor()` en lugar de `StructuredTaskScope`. Depender de una API preview en producción obliga a `--enable-preview` en cada JVM del despliegue y expone el sistema a cambios incompatibles entre versiones del JDK.

6. Estrategia de pruebas

```
./gradlew build -x :app:test      # unitarias de dominio + reglas de arquitectura
./gradlew :app:test              # integración: Postgres y Kafka reales (requiere Docker)
```

`OrderLifecycleIT` levanta PostgreSQL y Kafka con Testcontainers y recorre el ciclo completo —crear → outbox → relay → Kafka → pagar— verificando además que un reintento con la misma clave no cobra dos veces, que `created_at` no se reescribe y que la versión del optimistic locking avanza en cada actualización.

7. Cómo se extrae un microservicio

1. Se mueve `domain-X` y sus adaptadores a un repositorio nuevo.
2. Sus puertos ya existen; solo cambia la implementación (HTTP o Kafka en vez de JPA local).
3. Los eventos que ya publica por el outbox se convierten en su contrato público.

4. Las reglas ArchUnit garantizan que no había acoplamiento oculto que romper.

El coste de esta extracción es bajo *porque* las reglas se ejecutaron en cada commit, no porque el diagrama lo prometiera.

8. Limitaciones conocidas

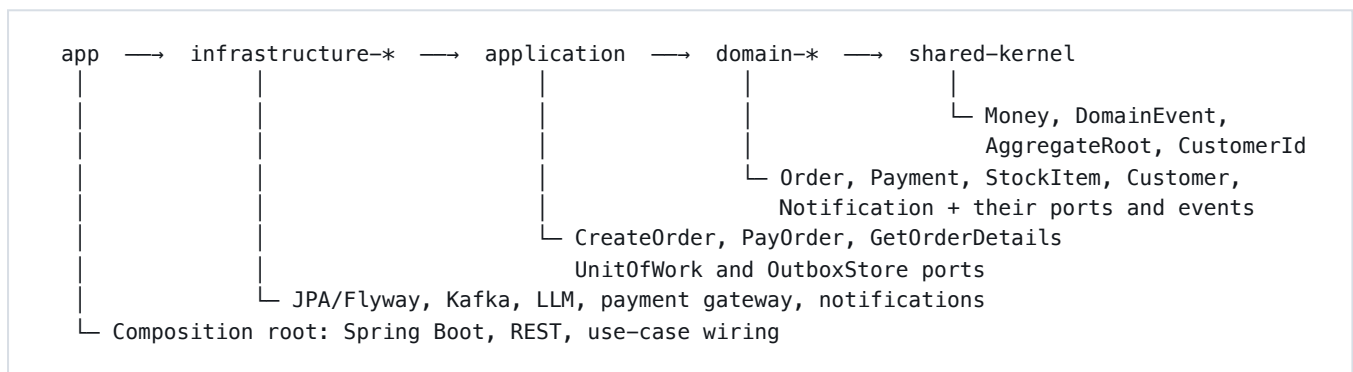
- **Deduplicación del consumidor en memoria.** `OrderEventsConsumer` usa un `Set` en memoria: se pierde al reiniciar y no se comparte entre réplicas. Debe respaldarse en base de datos antes de cualquier despliegue con más de una instancia.
- **Sin autenticación ni autorización.** No hay Spring Security ni OAuth2.
- **Sin trazabilidad distribuida.** Actuator expone métricas Prometheus, pero no hay OpenTelemetry ni correlación de trazas entre el relay y el consumidor.
- **Soft delete declarado pero no aplicado.** La columna `deleted` existe en `orders` y ninguna consulta la filtra.
- **El gateway de pago es falso.** `FakePaymentGateway` aprueba por umbral de importe; no hay reintentos, circuit breaker ni timeouts.

1. What this system is

A modular e-commerce monolith organised into **fourteen Gradle modules**. The design thesis is that any bounded context can be extracted as a microservice without rewriting its logic: the domain knows no framework, every outward call goes through a port, and contexts talk to each other through events rather than direct calls.

What separates this repository from a nice diagram is that **the build verifies those boundaries**: twenty-five ArchUnit rules fail the compilation if anyone crosses them.

2. Module map and the dependency rule



The arrows point one way only. `domain-*` imports neither Spring nor JPA nor Jakarta Validation; neither does `application`. The framework lives exclusively in `app` and `infrastructure-*`.

Why the application layer is framework-free too

This is the least conventional decision in the repository and the one with the widest consequences. A use case needs atomicity, and the reflex is to annotate it with `@Transactional` — which would tie the application layer to the framework. Instead, the transaction arrives through a port:

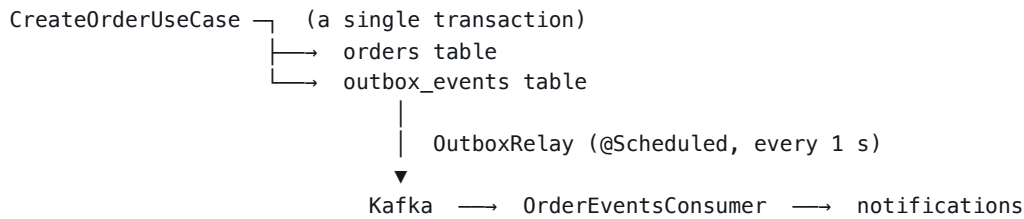
```
public interface UnitOfWork {
    <T> T execute(Supplier<T> work);
}
```

Infrastructure implements it with `TransactionTemplate`. The use case gets atomicity without importing a single Spring class, and the `applicationLayerDoesNotDependOnSpring` rule stops that from eroding over time.

3. The core: how consistency is guaranteed

3.1 Transactional outbox

A domain event is **never published directly to Kafka**. The use case writes the aggregate and its event in the same commit; a relay publishes it afterwards.



The problem it solves. Writing to the database and publishing to Kafka are two different systems. Done separately — the *dual-write* pattern — if Kafka fails right after the commit, the order exists but the event is gone for good, with no trace and no way to detect it. With the outbox the event sits in the same transaction as the order: either both are stored or neither is.

The relay marks `published = true` only once the broker acknowledges the write. If Kafka is down the row stays pending and is retried on the next cycle. This yields **at-least-once** delivery, which is exactly why the consumer deduplicates by `eventId`.

The publisher is deliberately annotated `@Transactional(propagation = MANDATORY)`: if anyone tries to publish an event outside an active transaction, the call fails rather than silently reintroducing the dual-write.

3.2 Payment idempotency

Charging is the least retryable operation in the system. The endpoint requires an `Idempotency-Key` header and the use case splits into three steps:

STEP	WHAT HAPPENS
Transaction 1	Reserve the key by creating the payment in <code>PENDING</code> . Uniqueness is enforced by the index on <code>payments.idempotency_key</code> , not by application code.
No transaction	Call the payment gateway. A network call must never hold a database transaction open.
Transaction 2	Record the outcome and the order's new state in a single commit.

```
curl -X POST localhost:8080/api/v1/orders/$ORDER_ID/payments \
-H "Idempotency-Key: $(uuidgen)"
```

Repeating the request with the same key returns the existing payment without calling the gateway again. Without this, a retry from the client or the load balancer charged twice.

4. Executable architecture rules

RULE	WHAT IT PREVENTS
<code>domainDoesNotDependOnSpring</code>	The domain becoming framework-bound and impossible to extract.
<code>applicationLayerDoesNotDependOnSpring</code>	Use cases filling up with annotations instead of ports.
<code>controllersDoNotAccessRepositories</code>	A controller bypassing the use case and talking to the database.
<code>onlyInfrastructureImplementsPorts</code>	An adapter appearing outside the infrastructure layer.
<code>noCyclicDependenciesBetweenModules</code>	Module cycles, which would make extracting a single one impossible.
<code>domainModulesDoNotDependOnEachOther</code>	Direct coupling between bounded contexts. Covers all 20 pairs .

A rule that passes vacuously is worse than no rule. The ArchUnit import fails loudly if it cannot analyse a single class — precisely because that was the most serious silent failure this repository had.

5. Decision: JDK 26 with bytecode 25

The build **compiles with JDK 26** (toolchain auto-provisioned via foojay) but emits **bytecode 25** through `options.release = 25`.

The reason is concrete and measurable: the ASM bundled with ArchUnit 1.4.1 only reads up to class file **V25**. Given V26 bytecode it discards every class *silently* — no error, no warning — and the architecture rules pass without checking anything. Working rules were preferred over a higher version number in the bytecode. The decision is revisited on every ArchUnit upgrade.

For the same reason production code uses no preview APIs: `GetOrderDetailsUseCase` parallelises with `Executors.newVirtualThreadPerTaskExecutor()` instead of `StructuredTaskScope`. Depending on a preview API in production forces `--enable-preview` on every JVM in the deployment and exposes the system to breaking changes between JDK releases.

6. Testing strategy

```
./gradlew build -x :app:test      # domain unit tests + architecture rules
./gradlew :app:test              # integration: real Postgres and Kafka (requires Docker)
```

`OrderLifecycleIT` starts PostgreSQL and Kafka with Testcontainers and walks the full cycle — create → outbox → relay → Kafka → pay — additionally verifying that a retry with the same key does not charge twice, that `created_at` is not rewritten, and that the optimistic-lock version advances on every update.

7. How a microservice gets extracted

1. Move `domain-X` and its adapters into a new repository.
2. Its ports already exist; only the implementation changes (HTTP or Kafka instead of local JPA).
3. The events it already publishes through the outbox become its public contract.

4. The ArchUnit rules guarantee there was no hidden coupling left to break.

The cost of that extraction is low *because* the rules ran on every commit, not because the diagram promised it.

8. Known limitations

- **In-memory consumer deduplication.** `OrderEventsConsumer` uses an in-memory `Set` : it is lost on restart and not shared across replicas. It must be backed by the database before any multi-instance deployment.
- **No authentication or authorisation.** Neither Spring Security nor OAuth2.
- **No distributed tracing.** Actuator exposes Prometheus metrics, but there is no OpenTelemetry and no trace correlation between the relay and the consumer.
- **Soft delete declared but not applied.** The `deleted` column exists on `orders` and no query filters on it.
- **The payment gateway is fake.** `FakePaymentGateway` approves by amount threshold; there are no retries, circuit breaker or timeouts.